

Day 10: Intro to Python Part 3 — Functions + Booleans

Today I moved to Python Part 3. I focused on functions (using `def`) and then started learning boolean expressions, relational operators, and boolean operators. I'm still learning, so I wrote everything in a way that makes it easy for me to come back later.

Functions (def = define)

The most important keyword I learned today was `def`. `def` stands for “define”, and for me it basically feels like: I'm creating a name for a block of code, so when I call that name later, the whole block runs.

Example idea: I want a function called `who_am_i` that introduces me. So instead of rewriting the intro every time, I can just call `who_am_i()`.

```
def who_am_i():
    name = "Mohammed"
    age = 16
    print("My name is " + name + " and I am " + str(age) + " years old")

who_am_i()
```

I learned a few important things from this example:

- The parentheses after the function name can be empty if the function doesn't take input.
- The colon (`:`) is required after `def ...()`.
- The code inside the function must be indented (usually 4 spaces).

Also, the variables I create inside the function (like `name` and `age` here) are local variables. That means they exist only inside the function, and once the function ends, I can't use them outside.

Function naming and underscores

About underscores: I noticed that a lot of Python functions use `snake_case` (words separated by underscores), like `who_am_i`. It's not forced, but it's the common style, and it makes names easier to read.

Functions with parameters (inputs)

Then I practiced functions that take input. The input goes inside the parentheses, and it becomes a parameter. For example, I made a function that adds 100 to a number.

```
def add_100(num):
    print(num + 100)

add_100(150)
```

In my head, num is like x in math. When I call `add_100(150)`, num becomes 150, then it prints 250.

Return (when I want a value back)

I also learned about `return`. `return` sends a value back from the function. One important thing I learned: any code after `return` inside the same function will never run.

```
def multiply(x, y):  
    return x * y  
  
print(multiply(7, 7))
```

I also learned something that confused me at first: if I call `multiply(7, 7)` by itself, it won't show anything on the terminal. To display it, I need to wrap it with `print()`, like `print(multiply(7, 7))`.

Boolean expressions (True / False)

After functions, I started boolean expressions. In Python, booleans are written as `True` and `False` (capital T/F).

```
bool1 = True  
bool2 = (3 * 3 == 9)  
bool3 = False  
bool4 = (3 * 3 != 9)  
  
print(bool1)  
print(bool2)  
print(bool3)  
print(bool4)
```

I learned that `==` is used to check equality, and `!=` means "not equal". So `3 * 3 == 9` is `True`, and `3 * 3 != 9` is `False`.

Checking the type (bool vs string)

One tricky part I noticed: if I put `True` inside quotes, it becomes a string, not a boolean. So I used `type()` to check.

```
bool5 = "True"  
print(type(bool1))  
print(type(bool5))
```

Relational and boolean operators

For relational and boolean operators, I realized the truth tables are the best way for me to study. I'm going to include a truth table screenshot in the proof section on Medium when I publish this day.

This was Day 10 for me. Functions made a lot of sense because it feels like I'm packaging code into a reusable name. And the boolean part was a good start — I'm still getting used to the operators, but

I'm getting there.